

R E L A Z I O N E T E C N I C A

CodeQuest

RPG Educativo di Programmazione

Studente	_____
Classe	_____
Materia	Informatica
Docente	_____
Anno Scolastico	2025 / 2026

Indice

1. Introduzione	3
2. Analisi del Progetto	4
3. Tecnologie Utilizzate	5
4. Architettura del Progetto	7
5. Analisi Dettagliata del Codice	9
6. Database	18
7. Interfaccia Grafica e UX	21
8. Sicurezza	23
9. Test e Funzionamento	24
10. Problemi Riscontrati e Soluzioni	25
11. Possibili Miglioramenti Futuri	26
12. Conclusioni	27

1. Introduzione

Descrizione generale

CodeQuest è un'applicazione web educativa che unisce narrazione interattiva, meccaniche di gioco RPG e apprendimento della programmazione. Il progetto è stato concepito per accompagnare gli studenti in un percorso guidato attraverso cinque linguaggi o ambienti informatici fondamentali, partendo dall'HTML e arrivando fino al C++, passando per JavaScript, PHP/MySQL e la shell Bash.

L'approccio scelto è quello della gamification: ogni argomento è presentato come un capitolo di un videogioco, in cui lo studente esplora una mappa, dialoga con personaggi non giocanti (NPC) che fungono da mentori, risolve puzzle di codice e affronta un boss finale. Il completamento di ogni sfida viene salvato sia localmente nel browser sia in un database remoto, consentendo al docente di monitorare in tempo reale i progressi della classe.

Obiettivo didattico

Il progetto nasce per rispondere a un problema concreto: come rendere l'apprendimento dei linguaggi di programmazione più coinvolgente per gli studenti. La soluzione adottata è presentare ogni concetto informatico all'interno di una narrazione con una villain chiamata ARA, un'intelligenza artificiale corrotta che minaccia il sistema digitale. Lo studente deve sconfiggerla risolvendo sfide di codice capitolo per capitolo.

Il percorso è deliberatamente progressivo: si parte dal markup e dal design (HTML e CSS), si passa alla logica applicativa (JavaScript), poi al backend e al database (PHP e MySQL), all'ambiente di sistema (MS-DOS e Bash) e infine ai linguaggi vicini all'hardware (C++ e C). Questa sequenza rispecchia il tipico curriculum di informatica di un istituto tecnico o professionale.

Tipologia di utenti

Studenti: accedono al gioco tramite registrazione con codice classe, completano missioni e puzzle, e visualizzano il proprio avanzamento nella dashboard personale.

Docenti: creano e gestiscono classi, distribuiscono il codice di accesso agli studenti e monitorano i progressi dell'intera classe attraverso la dashboard dedicata.

2. Analisi del Progetto

Funzionalità principali

Il progetto può essere suddiviso in tre aree funzionali distinte ma strettamente integrate:

- **Area di accesso e autenticazione:** comprende le pagine di selezione del ruolo, login e registrazione per studenti e docenti.
- **Area di gioco:** il motore Phaser 3 gestisce l'esplorazione delle mappe, i dialoghi con gli NPC, la risoluzione dei puzzle di codice e il combattimento con i boss.
- **Area gestionale:** le dashboard di studente e docente mostrano rispettivamente i progressi personali e quelli dell'intera classe.

Flusso generale dell'applicazione

L'utente apre il browser e visualizza la landing page (index.html). Da qui, cliccando su "Start Mission", viene reindirizzato alla pagina di selezione del ruolo (selection.html). In base alla scelta, accede al login studente o docente.

Dopo il login, lo studente viene indirizzato al gioco (game.html), mentre il docente accede alla propria dashboard gestionale. Nel gioco, la sequenza per ogni capitolo è sempre la stessa: esplorazione della mappa, dialogo con il mentore, completamento dei quattro puzzle di codice nell'ordine corretto, sblocco e sconfitta del boss, salvataggio del progresso e passaggio al capitolo successivo.

I cinque capitoli

Capitolo	Linguaggio	NPC Guida	Boss Finale	Puzzle
Capitolo 1	HTML + CSS	Mentore	Tag Overload	4 puzzle
Capitolo 2	JavaScript	Luna	Infinite Bug	4 puzzle
Capitolo 3	PHP + MySQL	Max	SQL Phantom	4 puzzle
Capitolo 4	MS-DOS + Bash	Ivy	Rootkit Prime	5 puzzle
Capitolo 5	C++ + C	Mentore + Luna + ARA::CORE	ARA CORE	4 puzzle

3. Tecnologie Utilizzate

Frontend — HTML, CSS, JavaScript

Tutto il frontend di CodeQuest è scritto in HTML5, CSS3 e JavaScript vanilla (ES6+). Non è stato utilizzato nessun framework frontend come React o Vue: la scelta è stata quella di lavorare direttamente con le API native del browser, il che rende il progetto più semplice da distribuire senza passi di compilazione.

I moduli JavaScript seguono la sintassi ES6 con import/export nativi. Questo approccio è possibile perché il progetto gira su un server locale Apache (XAMPP) che serve i file statici senza necessità di un bundler come Webpack. I fogli di stile CSS sono separati per area: landing.css, login.css, dashboard.css e code-editor.css.

I font usati nell'interfaccia grafica sono Orbitron (per i titoli e gli elementi principali) e Share Tech Mono (per testi a carattere tecnico e monospace). Entrambi sono caricati da Google Fonts e contribuiscono all'estetica cyberpunk/informatica del progetto.

Motore di gioco — Phaser 3

Phaser 3 è il framework open source per lo sviluppo di videogiochi 2D in JavaScript. Nel progetto viene usato nella versione 3.x caricata da CDN tramite il file proxy `js/engine/phaser.js`, che riesporta semplicemente il modulo.

Fisica arcade: Phaser mette a disposizione un sistema fisico semplificato chiamato "arcade". È stato scelto questo motore perché il tipo di gioco è top-down (visuale dall'alto senza gravità) e non richiede simulazioni fisiche complesse. La gravità è impostata a zero e le collisioni sono gestite tra il giocatore, i muri e le zone interattive.

Rendering pixel art: La configurazione `pixelArt: true` in `main.js` disabilita il filtraggio antialiasing, così i grafici generati proceduralmente mantengono i bordi netti tipici della pixel art retro.

Scale mode RESIZE: Il canvas Phaser si ridimensiona automaticamente con la finestra del browser grazie alla modalità `Phaser.Scale.RESIZE`, mantenendo sempre il contenuto centrato.

Backend — PHP

Il server-side è sviluppato interamente in PHP 7+. Sono stati scelti PHP e MySQL perché sono le tecnologie più diffuse nell'ambiente XAMPP, lo stack tipico degli istituti scolastici italiani, e perché rispecchiano i contenuti del Capitolo 3 del gioco stesso (PHP + MySQL).

Il backend non usa framework PHP (niente Laravel o Slim): tutte le API sono file PHP autonomi che ricevono richieste POST in formato JSON e rispondono con JSON. Questo approccio è volutamente semplice e leggibile per scopi didattici.

PDO (PHP Data Objects): la connessione al database usa PDO invece della vecchia estensione `mysqli`. PDO è preferibile perché supporta prepared statements in modo uniforme, riducendo il rischio di SQL injection, e permette di cambiare database con modifiche minime al codice.

Database — MySQL / MariaDB

Il database è MySQL (o MariaDB nella versione inclusa in XAMPP), un RDBMS relazionale open source. La scelta di MySQL è naturale per un progetto PHP scolastico. Lo schema è definito nel file `php/config/schema.sql` e comprende cinque tabelle: `teachers`, `classes`, `students`, `chapters` e `student_chapter_progress`.

Persistenza lato client — localStorage

Oltre al database server, il progetto usa il localStorage del browser per salvare lo stato della partita localmente. Questo permette la continuità del gioco anche prima che la sincronizzazione con il server sia completata, e offre una forma di fallback se la rete non è disponibile. I dati memorizzati includono l'ID studente, il nome, il capitolo corrente, i puzzle completati e lo stato dei boss.

Librerie e strumenti

Phaser 3	Framework per il motore di gioco 2D (caricato da CDN)
docx (npm)	Libreria usata per generare file Word (non fa parte del gioco)
Google Fonts	Font Orbitron e Share Tech Mono per l'interfaccia grafica
XAMPP	Stack locale Apache + PHP + MariaDB per lo sviluppo e il test
PDO	Astrazione PHP per l'accesso al database con prepared statements
ES6 Modules	Sistema di moduli JavaScript nativo, senza bundler
JSON	Formato di interscambio dati tra frontend JavaScript e backend PHP

4. Architettura del Progetto

Struttura delle cartelle

```
codeQuest-master/
├─ index.html           # Landing page
├─ selection.html       # Selezione ruolo
├─ student_login.html   # Login/registrazione studente
├─ teacher_login.html   # Login/registrazione docente
├─ student_dashboard.html # Dashboard studente
├─ teacher_dashboard.html # Dashboard docente
├─ game.html            # Contenitore del gioco Phaser
├─ assets/images/       # Sprite sheet e sfondi
├─ css/                 # Fogli di stile separati per area
├─ js/
│   ├─ main.js          # Entry point Phaser
│   ├─ auth/auth.js     # Sistema di autenticazione
│   ├─ engine/          # Motori di gioco (GameState, SaveSystem, ...)
│   ├─ scenes/          # Scena Boot, Menu, Chapter
│   ├─ scene/baseScene.js # Scena base con helper comuni
│   ├─ input/input.js   # Gestione input WASD
│   ├─ data/chapters/    # JSON dei 5 capitoli
│   ├─ data/maps_data.json # Layout delle mappe
│   ├─ data/assets_data.json
│   ├─ data/animations_data.json
│   └─ dialogs/         # JSON dei dialoghi cap0-cap4
└─ php/
    ├─ config/database.php # Connessione PDO singleton
    ├─ config/schema.sql   # Schema del database
    └─ api/               # 13 endpoint REST-like
```

Pattern architetturali

Singleton: GameState e SaveSystem sono oggetti singleton (letterali JavaScript). Vengono importati ovunque nel codice e mantengono un'unica istanza condivisa per tutta la sessione di gioco. Questo evita inconsistenze nello stato della partita.

Template Method (Scene): le scene Phaser (BootScene, MenuScene, ChapterScene) estendono BaseScene, che fornisce metodi helper riutilizzabili come createButton(), showNotification(), createSciFiBackground(). Le scene figlie implementano i metodi specifici (create(), update()) secondo il pattern Template Method.

Data-driven design: i contenuti di gioco (mappe, puzzle, dialoghi, NPC, animazioni) sono separati dal codice e caricati da file JSON a runtime. Questo permette di modificare testi, sfide e struttura dei capitoli senza toccare il codice JavaScript.

Event-based communication: MapLoader comunica con ChapterScene tramite eventi Phaser (`scene.events.emit/on`). Quando MapLoader legge la mappa e trova un'entità da spawnare, emette l'evento `spawn_entity`. ChapterScene lo ascolta e crea il relativo sprite. Questo disaccoppia il caricamento dati dalla logica di presentazione.

REST-like API: il backend PHP espone endpoint separati per ogni operazione. Ogni file PHP riceve un POST JSON e risponde con un oggetto JSON con i campi success (booleano) e message (stringa). Il frontend JavaScript usa fetch() per comunicare con questi endpoint.

5. Analisi Dettagliata del Codice

Pagine HTML dell'interfaccia web

index.html — Landing page

È la pagina di ingresso del progetto. La sua funzione è esclusivamente presentativa: mostra il logo testuale `</> CODEQUEST`, il sottotitolo "THE CODE AWAITS" e un unico pulsante di accesso "START MISSION" che porta alla selezione del ruolo. La semplicità è intenzionale: non c'è nessuna scelta da fare, solo un invito a entrare.

selection.html — Selezione del ruolo

Questa pagina mostra due card: una per l'accesso studente e una per l'accesso docente. La separazione avviene qui, prima ancora del login, in modo da portare l'utente direttamente nel percorso giusto. Questa pagina non ha logica JavaScript propria: ogni card è semplicemente un link verso `student_login.html` o `teacher_login.html`.

student_login.html e teacher_login.html — Autenticazione

Le due pagine di login seguono la stessa struttura: un'intestazione con logo e ruolo, due tab (LOGIN e REGISTRAZIONE) e i relativi form. La differenza sostanziale è che nel form di registrazione studente compare il campo Class Code, assente nel form docente. Entrambe le pagine usano lo stesso file `auth.js`, che adatta il comportamento in base all'attributo `data-role` del tag `body`.

student_dashboard.html — Dashboard studente

Questa pagina è il pannello di controllo personale dello studente. Dopo il login, qui si può vedere il nome utente, il progresso totale e lo stato dettagliato di ogni capitolo. La tabella mostra cinque colonne: Missione 1, Missione 2, Missione 3, Missione 4 e Boss Finale. Ogni cella è un indicatore visivo (completato / non completato). Il pulsante GIOCA avvia il gioco al capitolo corrispondente al punto di avanzamento dello studente.

teacher_dashboard.html — Dashboard docente

È la pagina gestionale più importante del progetto. Il docente può creare nuove classi, eliminare classi esistenti, selezionare una classe e visualizzare i progressi degli studenti. Sono presenti due modalità di visualizzazione: la vista riepilogo classe (una riga per studente, colonne per capitolo) e la vista singolo studente (dettaglio missione per missione). Questa pagina è di fatto il pannello amministrativo del progetto.

game.html — Contenitore del gioco

Questa pagina non contiene il gioco in sé, ma fornisce il contenitore DOM in cui Phaser renderizza il canvas. Gli elementi presenti sono l'area `#game-container`, un pulsante per tornare alla dashboard studente e l'import del file `main.js` come modulo ES6. La scelta di tenere `game.html` molto leggera è corretta: non deve interferire con il rendering di Phaser.

Sistema di autenticazione (auth.js)

Il file `js/auth/auth.js` gestisce sia il login sia la registrazione per entrambi i ruoli. Il sistema è organizzato come un oggetto letterale `Auth` con i metodi `init()`, `setupTabs()`, `setupForms()`, `validate()`, `handleLogin()`, `handleRegister()`, `saveSession()`, `updateStudentProgress()` e `showSuccess()`.

All'avvio, `init()` legge l'attributo `data-role` dal tag `body` per determinare il contesto (studente o docente). Questo permette di usare lo stesso file per le due pagine senza duplicare il codice.

I tab vengono collegati tramite `addEventListener` ai pulsanti con classe `login-tab`, che mostrano e nascondono i form corrispondenti.

La funzione `validate()` effettua controlli client-side su email (formato regex), password (lunghezza minima 6 caratteri), conferma password (per la registrazione) e codice classe (lunghezza minima 5 caratteri, solo per studenti). Questi controlli sono un primo livello di difesa: la validazione reale avviene poi anche lato server.

Dopo un login riuscito, `saveSession()` memorizza nel `localStorage` le informazioni dell'utente. Per i docenti salva `teacherId` e `teacherName`; per gli studenti salva `studentId`, `studentName` e `xp`. Subito dopo, `updateStudentProgress()` recupera i progressi più aggiornati dal database e li scrive in `localStorage` nella chiave `cq_save_data`, così quando il gioco si avvia ha già i dati sincronizzati.

Funzione `escapeHTML`: sia in `auth.js` sia nei file di dashboard è presente questa funzione di utilità che sostituisce i caratteri speciali HTML (`&`, `<`, `>`, `'`, `"`) con le relative entità HTML. Viene usata ogni volta che si inserisce contenuto dinamico nel DOM per prevenire attacchi XSS.

BootScene — Inizializzazione del gioco

`BootScene` è la prima scena che Phaser esegue. Il suo compito è preparare le risorse di base prima che il gioco sia visibile. Nel metodo `preload()` vengono caricati `bg_lab.png` (lo sfondo della mappa) e `tileset.png` (il foglio di sprite per le tile della mappa). Una barra di caricamento grafica mostra l'avanzamento tramite l'evento `load.progress`.

Il metodo `create()` è dichiarato `async` e svolge tre operazioni nell'ordine corretto. Prima controlla se in `localStorage` esiste la chiave `cq_student_id`: se sì, imposta `GameState.studentId` e chiama `await GameState.syncFromDb()` per caricare i dati aggiornati dal server. Se non c'è una sessione, prova a caricare un salvataggio locale con `SaveSystem.load()`. Poi chiama `AssetGenerator.generateAll(this)` per generare proceduralmente tutti gli asset grafici. Infine, dopo 500 ms, passa alla scena successiva `MenuScene`.

Questa sequenza garantisce che quando il menu appare i dati della sessione siano già stati caricati e che gli asset grafici siano pronti.

MenuScene — Menu principale

`MenuScene` estende `BaseScene` e costruisce l'interfaccia del menu principale attraverso quattro metodi privati: `_buildTitle()`, `_buildChapterNav()`, `_buildMenuButtons()` e `_buildFooter()`.

`_buildTitle()`: crea il titolo CODEQUEST con effetto pulse tramite tween Phaser, cerchi concentrici animati come sfondo e il sottotitolo THE CODE AWAITS.

`_buildChapterNav()`: mostra cinque badge orizzontali con il nome di ogni capitolo (HTML/CSS, JAVASCRIPT, PHP/SQL, BASH, C++) e un'icona testuale (`<>`, `{}`, `<?`, `$>`, `++`). I badge hanno colori diversi per ogni capitolo.

`_buildMenuButtons()`: controlla se esiste un salvataggio con `SaveSystem.hasSave()`. Se sì, mostra il pulsante CONTINUE GAME; altrimenti mostra NEW GAME. Se l'utente vuole avviare una nuova partita ma esiste già un salvataggio, compare una finestra di conferma per evitare cancellazioni accidentali.

`_buildFooter()`: mostra il nome dell'utente connesso letto da `localStorage` e i comandi di base del gioco.

ChapterScene — Gameplay principale

ChapterScene è la scena più complessa del progetto. Estende BaseScene e gestisce l'intero gameplay di un capitolo: esplorazione della mappa, NPC, puzzle di codice, boss e salvataggio del progresso.

Il metodo `init(data)` riceve l'indice del capitolo da avviare e inizializza tutte le variabili di stato della scena (lista NPC, lista puzzle, zona boss, contatore puzzle completati, ecc.).

Il metodo `create()` è `async` e segue una sequenza precisa:

- Genera gli asset procedurali con `AssetGenerator.generateAll()`.
- Configura i controlli WASD e il tasto E per l'interazione.
- Crea il giocatore con `_buildPlayer()`.
- Carica i dati del capitolo con `MapLoader.load()` (merge di `maps_data.json` e `chapterX.json`).
- Crea i gruppi fisici per NPC e puzzle con `physics.add.staticGroup()`.
- Costruisce la mappa con `MapLoader.build()`.
- Costruisce l'HUD con `_buildHUD()`.
- Inizializza `DialogSystem`, `CodeEditor` e `CombatSystem`.
- Controlla se il boss è già sbloccato con `_checkBossUnlock()`.
- Avvia il dialogo introduttivo con `_playIntroDialog()`.

Il sistema di sblocco del boss è semplice ed efficace: `_checkBossUnlock()` confronta `completedPuzzlesCount` con il numero totale di puzzle del capitolo. Quando coincidono, il boss diventa visibile e interagibile tramite un'animazione di fade-in e una notifica a schermo.

L'interazione con oggetti avviene nel metodo `update()` in ogni frame: se il giocatore è abbastanza vicino a un NPC, un puzzle o il boss, appare il prompt [E] INTERAGISCI. Premendo E, la scena controlla il tipo di entità e chiama il metodo appropriato (`_interactWithNPC`, `_interactWithPuzzle`, `_interactWithBoss`).

DialogSystem — Motore dei dialoghi

`DialogSystem` è un oggetto singleton esportato da `js/engine/DialogSystem.js`. Gestisce i dialoghi narrativi tra il giocatore e gli NPC.

Le caratteristiche principali sono: box di dialogo in stile terminale cyber, effetto typing (scrittura carattere per carattere), nome del parlante colorato in base al personaggio, avanzamento con la barra spazio, e supporto per callback di fine dialogo.

Il sistema assegna un colore specifico a ogni personaggio: Mentore è giallo (`fbbf24`), Luna è viola (`a855f7`), Max è arancione (`f97316`), Ivy è verde (`10b981`), ARA e ARA::CORE sono rossi (`ff1744`). Questi colori sono coerenti con la palette dell'intera interfaccia.

I dialoghi vengono caricati dai file JSON in `js/dialogs/` (`cap0.json`, `cap1.json`, ecc.). La separazione del testo dal codice è una buona pratica: permette di modificare o tradurre i dialoghi senza toccare la logica del gioco.

CodeEditor — Editor di codice integrato

`CodeEditor` è uno degli elementi più caratteristici del progetto. Quando lo studente interagisce con un puzzle, questo editor si apre come overlay a tutto schermo sopra il canvas di Phaser. L'interfaccia simula un IDE da terminale con estetica cyber.

L'editor mostra: un'area di testo per il codice con numeri di riga aggiornati dinamicamente, un'area output tipo terminale, e tre pulsanti EXECUTE, HINT e CLOSE.

Quando lo studente preme EXECUTE, il codice inserito viene inviato a `php/api/validate_puzzle.php` tramite `fetch()`. Il server risponde con `success: true/false` e una lista di errori. Il risultato viene mostrato nel terminale dell'editor con colori diversi (verde per

successo, rosso per errore). Se la validazione ha successo, viene chiamata la callback `onSuccess`, che segna il puzzle come completato in `GameState` e controlla se sbloccare il boss.

Il pulsante HINT mostra un suggerimento progressivo: al primo clic mostra il primo hint, al secondo il secondo, e così via. Ogni richiesta di suggerimento viene registrata in `GameState.hintsUsed`.

L'apertura dell'editor disabilita temporaneamente la tastiera di Phaser (`scene.input.keyboard.enabled = false`) per evitare che i tasti premuti nell'editor vengano catturati anche dal gioco.

GameState e SaveSystem — Gestione dello stato

`GameState` è il singleton centrale che mantiene lo stato della partita: `studentId`, `currentChapter`, `puzzlesCompleted` (oggetto con capitolo come chiave e array di indici puzzle come valore), `bossesDefeated` (array di indici capitolo), `araCorruption` (valore che sale durante il gioco) e `hintsUsed`.

Il metodo `save()` è `async` e fa due cose in sequenza: prima salva localmente con `SaveSystem.save()`, poi invia il progresso al server tramite `fetch()` verso `save_progress.php`. Questo approccio garantisce che il salvataggio locale sia immediato (nessuna latenza) e quello remoto avvenga in background. Se la rete non è disponibile, il salvataggio locale è comunque valido.

`syncFromDb()` è l'operazione inversa: recupera i progressi dal server e aggiorna sia lo stato in memoria sia il `localStorage`. Viene chiamata all'avvio in `BootScene` se l'utente è autenticato.

`SaveSystem` è separato da `GameState` per rispettare il principio di singola responsabilità: `GameState` conosce la logica di gioco, `SaveSystem` conosce solo la persistenza in `localStorage`.

MapLoader — Caricamento delle mappe

`MapLoader` carica e costruisce la mappa di ogni capitolo. Il metodo `load()` fa due `fetch` in parallelo: `maps_data.json` per il layout (griglia di tile) e `chapterX.json` per la narrativa (NPC, puzzle, dialoghi, boss). I risultati vengono uniti con `spread operator` e restituiti a `ChapterScene`.

Il metodo `build()` percorre la griglia del layout, legge il tipo di tile per ogni cella, e piazza l'elemento corrispondente: `floor` per il pavimento, `wall` per i muri, `npc_*` per gli NPC, `puzzle_node` per le zone puzzle, `boss_node` per il boss. Le entità dinamiche vengono emesse tramite l'evento `spawn_entity` che `ChapterScene` ascolta.

La separazione tra layout spaziale (`maps_data.json`) e contenuto narrativo (`chapterX.json`) è una scelta progettuale molto buona: permette di modificare la disposizione della mappa senza toccare i dialoghi e viceversa.

AssetGenerator — Generazione procedurale degli asset

Tutti i grafici del gioco (personaggi, ambienti, nemici, effetti visivi) sono generati proceduralmente tramite le API `canvas` di Phaser, senza usare file immagine esternamente. `AssetGenerator` è il coordinatore che chiama quattro moduli specializzati:

- **CharacterAssets:** genera i personaggi del giocatore e degli NPC (Mentore, Luna, Max, Ivy, ARA). Ogni personaggio è disegnato con forme geometriche colorate.
- **EnvironmentAssets:** genera le tile di ambiente (pavimento, muri, edifici).
- **EnemyAssets:** genera i boss e i nemici.

- **VfxAssets:** genera gli effetti visivi come le particelle e i nodi di puzzle.

La generazione procedurale è una scelta importante: permette di avere un gioco funzionante senza necessità di un art director o di file grafici esterni. Il codice stesso è il "pittore". Questo è particolarmente adatto a un progetto scolastico.

6. Database

Struttura generale

Il database si chiama codequest ed è definito nel file `php/config/schema.sql`. Utilizza il motore InnoDB con charset `utf8mb4`, che supporta correttamente tutti i caratteri Unicode incluse le emoji. Comprende cinque tabelle: `teachers`, `classes`, `students`, `chapters` e `student_chapter_progress`.

Tabella teachers

Campo	Tipo / Note
<code>teacher_id</code>	VARCHAR(20) — Chiave primaria, generata dal backend
<code>nome</code>	VARCHAR(50) — Nome del docente
<code>cognome</code>	VARCHAR(50) — Cognome del docente
<code>email</code>	VARCHAR(150) UNIQUE — Email, deve essere univoca
<code>password_hash</code>	VARCHAR(255) — Hash bcrypt della password

Un docente si identifica con email e password. Il `teacher_id` è una stringa generata al momento della registrazione (non un auto-increment numerico). L'email deve essere univoca: se si tenta di registrare due docenti con la stessa email il server restituisce un errore.

Tabella classes

Campo	Tipo / Note
<code>class_id</code>	INT AUTO_INCREMENT — Chiave primaria numerica
<code>class_code</code>	VARCHAR(20) UNIQUE — Codice univoco della classe
<code>teacher_id</code>	VARCHAR(20) — Chiave esterna verso <code>teachers.teacher_id</code>
<code>class_name</code>	VARCHAR(100) — Nome descrittivo della classe

Un docente può creare più classi. Ogni classe ha un `class_code` univoco generato automaticamente dal backend al momento della creazione: gli studenti usano questo codice per iscriversi alla classe corretta. Il vincolo di foreign key su `teacher_id` ha l'opzione `ON DELETE CASCADE`: se un docente viene eliminato, tutte le sue classi vengono eliminate di conseguenza.

Tabella students

Campo	Tipo / Note
<code>student_id</code>	VARCHAR(50) — Chiave primaria, generata dal backend
<code>nome</code>	VARCHAR(100) — Nome dello studente
<code>cognome</code>	VARCHAR(50) — Cognome dello studente
<code>email</code>	VARCHAR(150) UNIQUE — Email dello studente
<code>password_hash</code>	VARCHAR(255) — Hash bcrypt della password

Campo	Tipo / Note
<code>class_id</code>	INT — Chiave esterna verso <code>classes.class_id</code> , nullable

`class_id` è nullable: questo permette di gestire studenti non ancora assegnati a nessuna classe o il cui docente ha eliminato la classe. Il vincolo ON DELETE SET NULL garantisce che eliminando una classe gli studenti non vengano cancellati, ma solo de-associati.

Tabella chapters

Campo	Tipo / Note
<code>chapter_id</code>	INT AUTO_INCREMENT — Chiave primaria
<code>titolo</code>	VARCHAR(50) — Titolo del capitolo

La tabella `chapters` contiene dati statici inseriti dallo script SQL al momento dell'inizializzazione: cinque record corrispondenti ai cinque capitoli del gioco. Questi record servono come riferimento nelle foreign key della tabella dei progressi.

Tabella student_chapter_progress

Campo	Tipo / Note
<code>student_id</code>	VARCHAR(50) — FK verso <code>students</code> , parte della PK composita
<code>chapter_id</code>	INT — FK verso <code>chapters</code> , parte della PK composita
<code>mission_1_completed</code>	TINYINT(1) DEFAULT 0 — Missione 1 completata
<code>mission_2_completed</code>	TINYINT(1) DEFAULT 0 — Missione 2 completata
<code>mission_3_completed</code>	TINYINT(1) DEFAULT 0 — Missione 3 completata
<code>mission_4_completed</code>	TINYINT(1) DEFAULT 0 — Missione 4 completata
<code>boss_completed</code>	TINYINT(1) DEFAULT 0 — Boss finale sconfitto

Questa è la tabella più importante per il funzionamento del gioco. La chiave primaria è composita (`student_id`, `chapter_id`): ogni studente ha al massimo un record per capitolo. Le cinque colonne booleane (TINYINT(1)) tracciano lo stato di ogni missione e del boss. Non esiste una tabella separata per le missioni: il progresso è denormalizzato in colonne fisse. Questa scelta è semplice ed efficiente per il tipo di gioco realizzato, dato che il numero di missioni per capitolo è fisso.

Relazioni tra tabelle

La struttura relazionale segue una gerarchia chiara:

- Un teacher può avere più classes (relazione uno-a-molti).
- Una class può avere più students (relazione uno-a-molti).
- Ogni student può avere più record in `student_chapter_progress`, uno per ogni chapter (relazione molti-a-molti risolta con chiave primaria composita).

Le foreign key garantiscono l'integrità referenziale: non è possibile creare una classe senza un docente valido, né uno studente con un `class_id` inesistente.

7. Interfaccia Grafica e UX

Estetica generale

L'interfaccia di CodeQuest ha uno stile coerente e immediatamente riconoscibile: sfondi molto scuri (dal nero #020408 al blu scuro #0a1628), testo e accenti in colori neon (cyan #00d4ff, viola #a855f7, arancione #f97316, verde #10b981), bordi netti e font monospace. Questa estetica richiama ambienti informatici reali come terminali, IDE e ambienti di hacking etico.

Font Orbitron: usato per titoli, pulsanti e elementi di interfaccia. Ha un carattere geometrico, quasi fantascientifico, che si adatta perfettamente al tema tecnologico.

Font Share Tech Mono: usato per testi a carattere tecnico e output del terminale. È un font monospace che ricorda i terminali dei vecchi computer.

Landing page e accesso

La landing page è volutamente minimale: un unico pulsante, un logo e un sottotitolo. L'assenza di distrazioni guida l'utente immediatamente verso l'azione principale. La pagina di selezione ruolo usa due card grandi e ben separate, rendendo la scelta immediata anche su schermi piccoli.

Dashboard studente

La dashboard studente è progettata per essere leggibile a colpo d'occhio. La tabella dei progressi usa indicatori visivi (icone o colori) per distinguere le missioni completate da quelle ancora da fare. Il pulsante GIOCA è posizionato nell'header in modo da essere sempre visibile senza scorrere la pagina.

Dashboard docente

La dashboard docente è più articolata: prevede un sistema di filtri (selezione classe e modalità di visualizzazione) e due viste diverse. La separazione tra vista classe e vista singolo studente è una scelta UX importante: il docente può avere sia una panoramica rapida sia un'analisi dettagliata senza dover navigare su pagine diverse.

Interfaccia di gioco

Nell'interfaccia Phaser, la scena MenuScene crea un ambiente visivo ricco con animazioni (pulsazione del titolo, rotazione di cerchi concentrici, flickering del footer). Durante il gameplay in ChapterScene, il prompt di interazione [E] INTERAGISCI appare vicino agli elementi interattivi solo quando il giocatore è abbastanza vicino, evitando confusione visiva.

Il CodeEditor è l'elemento di interfaccia più elaborato: simula un IDE completo con numerazione di righe, evidenziazione del cursore, terminale output e status bar. Questa cura nei dettagli contribuisce a rendere credibile l'esperienza di "programmatore" che il gioco vuole trasmettere.

8. Sicurezza

Hashing delle password

Le password di studenti e docenti non vengono mai salvate in chiaro nel database. Al momento della registrazione, viene usata la funzione PHP `password_hash()` con l'algoritmo `bcrypt (PASSWORD_DEFAULT)`. Al login, `password_verify()` confronta la password inserita con l'hash memorizzato. Questa è la pratica standard per la gestione sicura delle credenziali.

Prepared statements e prevenzione SQL injection

Tutte le query al database usano prepared statements PDO. I dati ricevuti dal client non vengono mai concatenati direttamente nella stringa SQL: vengono passati come parametri separati tramite `execute($params)`. Questo rende impossibili gli attacchi di SQL injection, in cui un utente malintenzionato potrebbe inserire codice SQL nel campo email o password per manipolare il database.

Prevenzione XSS

La funzione `escapeHTML()` è presente sia in `auth.js` sia nei file di `dashboard`. Ogni volta che viene inserito contenuto dinamico nel DOM (nome utente, nome classe, ecc.) viene prima passato attraverso questa funzione che sostituisce i caratteri speciali HTML con entità sicure. Questo previene gli attacchi XSS (Cross-Site Scripting) in cui un utente potrebbe inserire codice HTML o JavaScript nel proprio nome per compromettere la dashboard.

Validazione a doppio livello

La validazione avviene sia lato client (`auth.js` controlla formato email, lunghezza password, codice classe) sia lato server (le API PHP verificano la presenza dei campi obbligatori, l'unicità dell'email, la validità del codice classe). La validazione client-side migliora l'esperienza utente con feedback immediato; quella server-side è quella che conta davvero per la sicurezza, perché non può essere aggirata modificando il codice JavaScript nel browser.

Verifica di proprietà prima delle operazioni critiche

Prima di eliminare una classe, `delete_class.php` verifica che la classe appartenga al docente che ha effettuato la richiesta (confrontando il `teacher_id` ricevuto con quello associato alla classe nel DB). Questo impedisce a un docente di eliminare le classi di un altro.

Limiti di sicurezza

Il progetto presenta alcune limitazioni di sicurezza tipiche di applicazioni scolastiche. La gestione della sessione avviene principalmente tramite `localStorage`: l'ID utente viene letto dal browser e inviato nelle richieste API senza un sistema di token firmati (come JWT) o sessioni server-side robuste. Ciò significa che un utente con accesso al browser potrebbe teoricamente modificare il proprio `studentId`. Per un ambiente scolastico controllato questo non è un problema grave, ma in un contesto di produzione reale sarebbe necessario implementare sessioni server-side o token di autenticazione firmati.

9. Test e Funzionamento

Requisiti di sistema

- Server web con supporto PHP 7.4 o superiore (es. XAMPP con Apache).
- Database MySQL 5.7 o MariaDB 10.3 o superiore.
- Browser moderno con supporto ES6 modules (Chrome, Firefox, Edge, Safari recenti).
- Connessione localhost (non è necessaria una connessione internet per il funzionamento base).

Procedura di installazione e avvio

Per avviare il progetto in ambiente locale:

- **1.** Copiare la cartella codeQuest-master/ nella directory htdocs di XAMPP (o equivalente).
- **2.** Avviare Apache e MySQL dal pannello di controllo XAMPP.
- **3.** Aprire phpMyAdmin e importare lo schema: eseguire il contenuto di `php/config/schema.sql` oppure da riga di comando con `mysql -u root -p < php/config/schema.sql`.
- **4.** Aprire il browser e navigare a `http://localhost/codeQuest-master/index.html`.

Flusso di test tipico

- Dalla landing page cliccare START MISSION.
- Scegliere TEACHER ACCESS e registrare un account docente.
- Annotare il class code generato automaticamente.
- Tornare alla selezione, scegliere STUDENT LOGIN e registrare uno studente con il class code appena ottenuto.
- Fare login come studente: il sistema reindirizza automaticamente a `game.html`.
- Nel gioco, esplorare la mappa con WASD, avvicinarsi al nodo puzzle e premere E.
- Nell'editor, scrivere la soluzione e premere EXECUTE.
- Completati tutti i puzzle, il boss si sblocca. Interagire con il boss per completare il capitolo.
- Tornare alla `student_dashboard` per verificare che il progresso sia stato salvato.
- Fare login come docente e verificare che il progresso dello studente sia visibile nella `teacher_dashboard`.

10. Problemi Riscontrati e Soluzioni Adottate

Sincronizzazione stato tra client e server

Un problema tipico nelle applicazioni con salvataggio ibrido (localStorage + database) è garantire che i due livelli siano sempre allineati. La soluzione adottata è una strategia semplice ma efficace: all'avvio (BootScene), se c'è una sessione attiva, si esegue sempre una sincronizzazione forzata dal database prima di mostrare il menu. In questo modo lo studente non vede mai dati obsoleti.

Gestione dei moduli ES6 senza bundler

L'uso di ES6 modules nativi richiede che i file siano serviti da un server HTTP (non possono essere aperti come file locali con file://). Questo è un vincolo noto che viene risolto richiedendo l'uso di XAMPP. Per facilitare la comprensione, il progetto include commenti che spiegano ogni import.

Compatibilità della tastiera nell'editor di codice

Quando l'editor di codice è aperto, i tasti premuti dall'utente per scrivere il codice non devono essere catturati anche da Phaser (che userebbe WASD per muovere il personaggio). La soluzione è disabilitare la tastiera Phaser con `scene.input.keyboard.enabled = false` all'apertura dell'editor e riattivarla alla chiusura.

Struttura delle missioni nel database

Il numero di missioni per capitolo (quattro puzzle più un boss) è fisso e hardcodato sia nel codice sia nel database. Se in futuro si volesse un numero variabile di puzzle per capitolo, la struttura attuale non lo supporterebbe senza modifiche allo schema. Per ora la scelta è stata privilegiare la semplicità rispetto alla flessibilità.

Sistema di combattimento semplificato

Nel codice di ChapterScene il sistema di combattimento è dichiarato come oggetto placeholder: `CombatSystem.start()` chiama direttamente la callback di vittoria senza una vera fase di combattimento interattivo. Questa è una limitazione consapevole: lo sviluppo del combattimento richiederebbe molto tempo e la priorità è stata completare il sistema di progressione e le sfide di codice.

11. Possibili Miglioramenti Futuri

Sistema di combattimento interattivo

Il punto di miglioramento più immediato è implementare un vero sistema di combattimento per i boss. L'idea del progetto (combattimento basato su snippet di codice) è già descritta nel README: invece di cliccare semplicemente su un boss, lo studente dovrebbe affrontare una sfida di codice più complessa o a tempo per sconfiggerlo.

Sessioni server-side

Sostituire la gestione delle sessioni basata su localStorage con un sistema di token JWT o sessioni PHP server-side renderebbe l'applicazione molto più robusta dal punto di vista della sicurezza. Questo è un miglioramento importante se il progetto venisse deployato in un ambiente reale accessibile da internet.

Editor con syntax highlighting

L'attuale editor di codice non evidenzia la sintassi del linguaggio. Integrare una libreria come CodeMirror o Monaco Editor (lo stesso usato da VS Code) offrirebbe un'esperienza di programmazione molto più vicina a quella di un IDE reale.

Statistiche avanzate per il docente

La dashboard docente potrebbe essere arricchita con grafici di avanzamento (quanti studenti hanno completato ogni capitolo), indicatori di difficoltà (quali puzzle ricevono più richieste di hint), e timeline dei progressi. Questi dati potrebbero aiutare il docente a identificare gli argomenti che creano più difficoltà.

Numero variabile di missioni per capitolo

Modificare lo schema del database per usare una tabella missions separata (con mission_id, chapter_id e nome) permetterebbe di avere un numero arbitrario di missioni per capitolo, rendendo il sistema molto più flessibile e facile da estendere.

Deploy su server pubblico

Attualmente il progetto funziona solo in localhost. Un passo evolutivo naturale sarebbe il deploy su un server pubblico (ad esempio su un hosting condiviso o una VPS), così tutta la classe potrebbe accedere senza necessità di installare XAMPP su ogni computer.

12. Conclusioni

Valutazione complessiva del progetto

CodeQuest è un progetto tecnicamente solido e didatticamente ben strutturato. Non si limita a mostrare pagine web statiche: realizza un sistema completo con autenticazione, ruoli, motore di gioco, progressione, database relazionale e API. Ogni componente è motivato da esigenze concrete e implementato con consapevolezza delle scelte effettuate.

La coerenza tra forma e contenuto è uno dei punti di forza più evidenti: il progetto insegna programmazione attraverso la programmazione stessa. L'estetica da terminale e IDE non è decorativa, ma narrativa: lo studente si immerge in un ambiente informatico e impara giocando in esso.

Tecnologie toccate

Il progetto permette di toccare concretamente molteplici ambiti dell'informatica moderna:

- Markup e stili (HTML5, CSS3, font esterni).
- Programmazione lato client (JavaScript ES6, moduli, fetch API, localStorage).
- Framework di gioco (Phaser 3, scene, fisica, asset procedurali).
- Programmazione lato server (PHP, PDO, JSON API).
- Database relazionale (MySQL, schema, foreign key, CRUD).
- Sicurezza applicativa (bcrypt, prepared statements, XSS prevention).
- Architettura software (singleton, template method, data-driven design, event-driven).

Competenze sviluppate

La realizzazione di CodeQuest ha richiesto la capacità di progettare un sistema multi-livello in cui frontend, motore di gioco e backend comunicano in modo coerente. Ha richiesto l'uso responsabile delle API di sicurezza (hashing password, query parametrizzate), la separazione dei contenuti dalla logica tramite file JSON, e la gestione della persistenza su due livelli (locale e remota).

Considerazioni finali

Il progetto dimostra come sia possibile, anche in un contesto scolastico, realizzare un'applicazione completa e funzionante che copra l'intero stack tecnologico. I limiti presenti (sistema di combattimento semplificato, sessioni client-side) sono limiti consapevoli, non errori progettuali: sono aree di miglioramento identificate chiaramente che definiscono una roadmap realistica per versioni future.

Fine relazione — CodeQuest — Anno Scolastico 2025/2026